A Generalized Quantum Inner Product and Applications to Financial Engineering

Vanio Markov[1], Charlee Stefanski[1,2], Abhijit Rao[1], and Constantin Gonciulea[1]

[1]Wells Fargo

[2]University of California, Berkeley

arXiv:2201.09845v1 [quant-ph] 24 Jan 2022

In this paper we present a canonical quantum computing method to estimate the weighted sum $\sum_{k=0}^{2^n-1} w_k f(k)$ of the values taken by a discrete function $f : \{0, \ldots, 2^n - 1\} \to \{0, \ldots, 2^m - 1\}$ for n, m positive integers and weights $w_k \in \mathbb{R}$ for $k \in \{0, \ldots, 2^n - 1\}$. The canonical aspect of the method comes from relying on a single linear function encoded in the amplitudes of a quantum state, and using register entangling to encode the function f.

We further expand this framework by mapping function values to hashes in order to estimate weighted sums of hashed function values $\sum_{k=0}^{2^n-1} w_k h_{f(k)}$, with $h_v \in \mathbb{R}$ for $v \in \{0, \ldots, 2^m - 1\}$. This generalization allows the computation of restricted weighted sums such as value at risk, comparators, as well as Lebesgue integrals and partial moments of statistical distributions.

We also introduce essential building blocks such as efficient encodings of standardized linear quantum states and normal distributions.

1 Introduction

Key areas of Financial Engineering such as Risk Management, Portfolios Optimization, Financial Data Analysis, and Machine Learning use methods of Linear Algebra and Fourier Analysis. A key operation in these areas is the inner product of two vectors - the sum of the products of their 'inner' corresponding components. Since the financial analysis is usually performed in very large vector spaces the efficient implementation of the inner product is critical for the feasibility of the Quantitative Financial Technology.

If a financial problem is modeled through the state space of a quantum system, then the inner product leverages quantum parallelism of quantum transformations. Quantum parallelism, together with interference, allows quantum algorithms such as Grover's search [1] and quantum Fourier transform [2] to achieve proven speedup compared to their classical counterparts.

In this paper we make the following contributions:

1. A generalized inner product that uses both quantum state and amplitude encoding
2. Standardized building blocks as a unifying framework for calculating weighted sums
3. Applications to weighted sum computations that occur in financial engineering, e.g. expected values, value at risk (VaR), derivative pricing, etc.
4. A canonical method for computing the expected value of discrete functions
5. Efficient methods for approximate and exact encodings of normal distributions
6. Efficient methods for approximate and exact encodings of linear functions, including a canonical identity function used in expected value computations

The canonical approach for computing quantum inner products leverages our prior work on encoding a binary polynomial representing a discrete function [3, 4], by adding weights to both its inputs and outputs. In this paper we will refer to polynomials of binary variables as binary polynomials.

In some of the inner product applications, we use circuits that efficiently encode linear and rational functions, and normal distributions for a given number of qubits.

The paper is organized as follows:

Section 2 introduces mathematical concepts and notation used throughout the paper, including details about encoding binary polynomials representing discrete functions.

Section 3 presents the methods we propose for computing inner products.

Section 4 discusses the preparation of quantum states representing standard distributions and functions. In particular, we propose methods for exact or approximate encoding of the raised cosine and the normal distributions, as well as linear functions.

Section 5 presents applications of the proposed framework.

Section 6 presents results of execution of some of the algorithms on IBM Q hardware [5].

Section 7 includes references to similar work.

Section 8 contains concluding remarks.

Appendix B contains a catalog of efficient circuits for exact encoding of linear quantum states and normal distributions.

2 Preliminaries

This section contains some mathematical background necessary to understand the main results of this paper.

2.1 Discrete Functions As Binary Polynomials

As stated in [6], any function $f : \{0, \ldots, 2^n - 1\} \to \mathbb{C}$, for a given integer $n > 0$, can be expressed as a polynomial $p : \{0,1\}^n \to \mathbb{C}$:

$$p(x_0, \ldots, x_{n-1}) = \sum_{k=0}^{2^n-1} f(k) p_k(x_0, \ldots, x_{n-1})$$

where p_k for $0 \leq k < 2^n$ are polynomials that satisfy the property

$$p_k(x_0, \ldots, x_{n-1}) = \begin{cases} 1, & \text{if } x_j = k_j \text{ for all } 0 \leq j < n \\ 0, & \text{otherwise} \end{cases}$$

with $k = \sum_{j=0}^{n-1} k_j 2^j$ being the binary expansion of k, with binary digits $k_j \in \{0,1\}$, for $0 \leq j < n$.

Therefore,

$$f(k) = p(k_0, \ldots, k_{n-1}).$$

The polynomials p_k for $0 \leq k < 2^n - 1$ can be defined by

$$\begin{aligned} p_k(x_0, \ldots, x_{n-1}) &= \left(\prod_{\substack{j \\ k_j=1}} x_j \right) \left(\prod_{\substack{j \\ k_j=0}} (1 - x_j) \right) \\ &= \prod_{j=0}^{n-1} ((2k_j - 1)x_j + 1 - k_j) \end{aligned} \tag{1}$$

Recall that the square of a binary value $x \in \{0,1\}$ is the value itself ($x^2 = x$), because $0^2 = 0$ and $1^2 = 1$.

2.2 Geometric Sequence State

For a quantum register with $m > 0$ qubits and an angle $\theta \in \mathbb{R}$ let us define the state

$$\begin{aligned} |\gamma_\theta\rangle &= \frac{1}{\sqrt{M}} \sum_{k=0}^{M-1} e^{ik\theta} |k\rangle_m \\ &= \frac{1}{\sqrt{M}} \sum_{k=0}^{M-1} (\cos(k\theta) + i \sin(k\theta)) |k\rangle_m \end{aligned} \tag{2}$$

where $M = 2^m$. Note that the amplitudes of this quantum state form a geometric sequence. This is one of the building blocks–referred to as a complex exponential signal–in Digital Signal Processing. In quantum computing this state plays an important role in the phase estimation and other algorithms, where the period of the signal represented by the amplitudes encodes some relevant information.

This state can be prepared with the circuit in Fig. 1, where P is the phase single qubit gate.

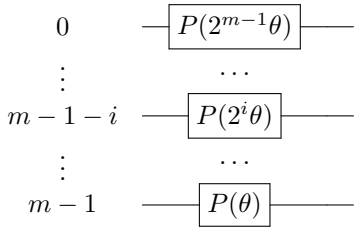

Figure 1: The quantum circuit for encoding the state represented in Eq. 2 in an m-qubit register. The circuit performs a series of Phase gates, denoted by P, using multiples of a given angle $\theta \in \mathbb{R}$.

2.3 Integer Encoding

Given a quantum register with $m > 0$ qubits and $M = 2^m$. To encode a given integer value $-\frac{M}{2} \leq j < \frac{M}{2}$, we first create the state $|\gamma_{j\frac{2\pi}{M}}\rangle$, as described in Section 2.2, and then apply the inverse Fourier transform operator ($QFT^\dagger$) to this state, resulting in the state $|j\rangle$ if $j \geq 0$ or $|j + M\rangle$ if $j < 0$, representing the Two's Complement representation of j, as described in [3, 4].

2.4 Binary Polynomial Encoding

Assume we have two quantum registers, a key register $|k\rangle_n$ with $n > 0$ qubits, and a value register $|v\rangle_m$ with $m > 0$ qubits. We'll use the notations $N = 2^n$ and $M = 2^m$.

For a binary polynomial $p : \{0,1\}^n \to \{0,1\}^m$ with integer coefficients we can use the Quantum Dictionary pattern, as introduced in [3, 4], to encode key-value pairs $(k, p(k))$, with $0 \leq k < N$ and $-\frac{M}{2} \leq p(k) < \frac{M}{2}$, in the key and value registers, respectively, by using a controlled version of the value encoding procedure described in Section 2.3.

The registers are entangled in such a way that when measured, the outcome will consist of a key and its

corresponding value. The key register $|k\rangle_n$ represents the inputs of the polynomial p, and the value register $|v\rangle_m$ represents the outputs.

Here is a brief overview of the implementation. We start by putting both registers in equal superposition:

$$\frac{1}{\sqrt{N}}\sum_{k=0}^{N-1}|k\rangle_n\frac{1}{\sqrt{M}}\sum_{v=0}^{M-1}|v\rangle_m \tag{3}$$

Any binary polynomial p of n variables with integer coefficients can be expressed as a sum of monomials:

$$p(x_0,\ldots,x_{n-1}) = \sum_{J\subseteq\{0,\ldots,n-1\}} c_J\prod_{j\in J}x_j \tag{4}$$

For a monomial $c_J\prod_{j\in J}x_j$ we encode the integer c_J using the value encoding procedure controlled on the qubits corresponding to J [4]. The resulting state is

$$\frac{1}{\sqrt{N}}\sum_{k=0}^{N-1}|k\rangle_n|\gamma_{p(k)\frac{2\pi}{M}}\rangle_m. \tag{5}$$

Finally, we apply the inverse Fourier transform operator to the value register and end up with the state

$$\frac{1}{\sqrt{N}}\sum_{k=0}^{N-1}|k\rangle_n|p(k)\rangle_m. \tag{6}$$

3 Inner Product Computation Methods: Simple and Generalized

Let V be the vector space describing an n-qubit quantum system. The inner product of two quantum states, $|\psi_A\rangle$ and $|\psi_B\rangle$, is defined as

$$\langle\psi_B|\psi_A\rangle = \sum_{i=0}^{N-1}\bar{b}_i a_i \tag{7}$$

where $N = 2^n$ and the states $|\psi_A\rangle$ and $|\psi_B\rangle$ are prepared by the unitary transformations A and B:

$$\begin{aligned}|\psi_A\rangle &= A|0\rangle_n = \sum_{i=0}^{N-1}a_i|i\rangle_n\\ |\psi_B\rangle &= B|0\rangle_n = \sum_{i=0}^{N-1}b_i|i\rangle_n\end{aligned} \tag{8}$$

3.1 Inner Product of Quantum States Using State Preparation Operators

The inner product of quantum states is invariant to unitary transformations, therefore we can apply the state preparation unitary B and represent $\langle\psi_B|\psi_A\rangle$ as

$$\langle\psi_B|\psi_A\rangle = \langle\psi_B|BB^\dagger|\psi_A\rangle \tag{9}$$

From Equation 8 we have

$$\langle\psi_B|B = \langle 0|_n,$$

and therefore the inner product can be expressed as

$$\langle\psi_B|\psi_A\rangle = \langle 0|B^\dagger A|0\rangle.$$

If the quantum state created by the circuit in Fig. 2 is

$$B^\dagger A|0\rangle_n = \gamma_0|0\rangle_n + \sum_{i=1}^{N-1}\gamma_i|i\rangle_n$$

then the amplitude of the state $|0\rangle_n$ is the inner product we want to calculate:

$$\langle\psi_B|\psi_A\rangle = \gamma_0. \tag{10}$$

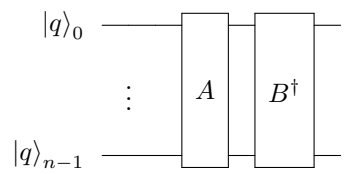

Figure 2: The quantum circuit that computes the inner product represented in Eq. 7 using the unitary operators A and $B^\dagger$, as defined in Eq. 8.

Let us consider the following problem context and solution for the computation of weighted sums using inner product:

Weighted Sum. *Given an integer $n > 0$ and $N = 2^n$, weights $w_k \in \mathbb{R}$ defined for integers $0 \le k < N$ and a function $f : \{0,\ldots,N-1\} \to \mathbb{R}$ we are interested in calculating the weighted sum*

$$\sum_{k=0}^{N-1}w_k f(k).$$

Solution. Let A be an operator that prepares a state $\sum_{k=0}^{N-1}a_k|k\rangle_n$, with $a_k = aw_k$, for $0 \le k < N$, where $a \in \mathbb{R}$ is a common factor, and let B be an operator that prepares the state $\sum_{k=0}^{N-1}b_k|k\rangle_n$ with $b_k = bf(k)$, for $0 \le k < N$, where b is a common factor.

Using the circuit represented in Fig. 2, we can compute the inner product $\sum_{k=0}^{N-1}a_k b_k$ as

$$E_{|0\rangle} := \langle 0|B^\dagger A|0\rangle_n = \sum_{k=0}^{N-1}a_k b_k = ab\sum_{k=0}^{N-1}w_k f(k),$$

therefore,

$$\sum_{k=0}^{N-1}w_k f(k) = \frac{1}{ab}E_{|0\rangle}$$

where $E_{|0\rangle} = \langle 0| B^\dagger A |0\rangle_n$ is the amplitude of $|0\rangle_n$ at the end of the computation.

The desired weighted sum is the amplitude of the initial state $|0\rangle_n$ after applying the unitary operator $B^\dagger A$ to it, which can be estimated using amplitude estimation algorithms. □

3.2 Generalized Inner Product

Let us consider two quantum registers, key and value, with n and m qubits and $N = 2^n, M = 2^m$ computational states, respectively. Given a function $f : \{0, \ldots, N-1\} \to \{0, \ldots, M-1\}$, we entangle the two registers using the quantum dictionary pattern discussed in Section 2.4.

Let $|\psi_A\rangle_n$ be the state of the key register prepared by the unitary operator A:

$$|\psi_A\rangle = A|0\rangle_n = \sum_{k=0}^{N-1} a_k |k\rangle_n .$$

The state of the two registers is

$$(A \otimes I_m)|0\rangle_{n+m} = |\psi_A\rangle |0\rangle_m = \sum_{k=0}^{N-1} a_k |k\rangle_n |0\rangle_m .$$

The unitary operator F entangles the two registers leading to the state

$$\begin{aligned} F(A \otimes I_m)|0\rangle_{n+m} &= F\left(\sum_{k=0}^{N-1} a_k |k\rangle_n |0\rangle_m\right) \\ &= \sum_{k=0}^{N-1} a_k |k\rangle_n |f(k)\rangle_m . \end{aligned} \tag{11}$$

Let $|\psi_B\rangle$ be the state of the value register prepared by the unitary operator B:

$$|\psi_B\rangle = B|0\rangle_m = \sum_{v=0}^{M-1} b_v |v\rangle_m .$$

When applying Hadamard gates to the qubits in the key register, we obtain the following state for the two registers:

$$(H^{\otimes n} \otimes B)|0\rangle_{n+m} = \frac{1}{\sqrt{N}} \sum_{\substack{0 \le k < N \\ 0 \le v < M}} b_v |k\rangle_n |v\rangle_m \tag{12}$$

Applying $(H^{\otimes n} \otimes B^\dagger)$ to the state represented in Equation 11 results in

$$(H^{\otimes n} \otimes B^\dagger) F (A \otimes I_m)|0\rangle_{n+m} .$$

Therefore,

$$\langle 0|_{n+m} (H^{\otimes n} \otimes B^\dagger) F (A \otimes I_m)|0\rangle_{n+m} = \sum_{k=0}^{N-1} a_k b_{f(k)} \tag{13}$$

The inner product of the amplitudes of the states in Equations 11 and 12 is the amplitude of the computational state $|0\rangle_{n+m}$. We refer to the right hand side of Equation 13 as a generalized inner product.

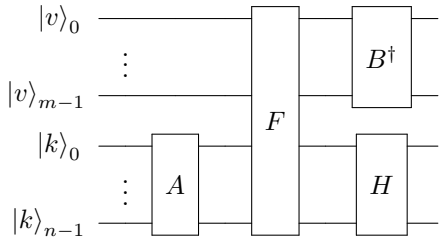

Figure 3: The quantum circuit for computing generalized inner products. The operator A encodes a given distribution of weights on the key register $|k\rangle_n$, and the operator F entangles the inputs and outputs of a function. Applying $B\dagger$ to the value register $|v\rangle_m$ and $H^{\otimes n}$ to the key register $|k\rangle_n$, results in a state which contains the desired inner product in the amplitude $|0\rangle_{n+m}$.

Let us consider the following problem context and solution for the computation of weighted sums using generalized inner products:

Controlled Weighted Sum. *Given integers $n, m > 0$ and $N = 2^n, M = 2^m$, weights $w_k \in \mathbb{R}$ defined for integers $0 \le k < N$, weights/hashes $h_v \in \mathbb{R}$ defined for integers $0 \le v < M$ and a function $f : \{0, \ldots, N-1\} \to \{0, \ldots, M-1\}$, we are interested in calculating the weighted sum of weighted/hashed function values,*

$$\sum_{k=0}^{N-1} w_k h_{f(k)}.$$

Solution. Let A be an operator that prepares a state $\sum_{k=0}^{N-1} a_k |k\rangle_n$, with $a_k = a w_k$, for $0 \le k < N$, where $a \in \mathbb{R}$ is a common factor, and let B be an operator that prepares the state $\sum_{v=0}^{M-1} b_v |v\rangle_m$ with $b_v = b h(v)$, for $0 \le v < M$, where $b \in \mathbb{R}$ is a common factor, and F an operator that encodes the function f.

Using the procedure described above, and represented in the circuit in Fig. 3, we can compute the inner product of the states in Equations 11 and 12:

$$\begin{aligned} E_{|0\rangle} &:= \langle 0|_{n+m} (H^{\otimes n} \otimes B^\dagger) F (A \otimes I_m)|0\rangle_{n+m} \\ &= \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} a_k b_{f(k)} \\ &= \frac{ab}{\sqrt{N}} \sum_{k=0}^{N-1} w_k h_{f(k)}. \end{aligned}$$

Therefore,

$$\sum_{k=0}^{N-1} w_k h_{f(k)} = \frac{\sqrt{N}}{ab} E_{|0\rangle}$$

with $E_{|0\rangle} = \langle 0|_{n+m} (H^{\otimes n} \otimes B^\dagger) F (A \otimes I_m) |0\rangle_{n+m}$ being the amplitude of $|0\rangle_{n+m}$ at the end of the computation.

The desired weighted sum is the amplitude of the initial state $|0\rangle_{n+m}$ after applying the unitary operator $(H^{\otimes n} \otimes B^\dagger) F (A \otimes I_m)$ to it, which can be estimated using amplitude estimation algorithms. □

Expected Value. *If B is the operator L_m that encodes h as the identity, i.e. $h(v) = v$ for $0 \leq v < M$, and $b = \frac{1}{\sqrt{\sum_{v=0}^{M-1} v^2}} = \sqrt{\frac{6}{(M-1)M(2M-1)}}$ we obtain a canonical way to compute the expected value of f:*

$$\sum_{k=0}^{N-1} w_k f(k) = \frac{\sqrt{N}}{a} \sqrt{\frac{(M-1)M(2M-1)}{6}} E_{|0\rangle}$$

with $E_{|0\rangle} = \langle 0| (H^{\otimes n} \otimes L_m^\dagger) F (A \otimes I_m) |0\rangle_{n+m}$ being the amplitude of $|0\rangle_{n+m}$ at the end of the computation.

Mean Value. *With the notations in the* **Expected Value** *context, if $A = H^{\otimes n}$, then $w_k = 1$ for $0 \leq k < N$ and $a = \frac{1}{\sqrt{N}}$, and we get the mean value of f:*

$$\frac{1}{N} \sum_{k=0}^{N-1} f(k) = \sqrt{\frac{(M-1)M(2M-1)}{6}} E_{|0\rangle}$$

with $E_{|0\rangle} = \langle 0| (H^{\otimes n} \otimes L_m^\dagger) F (H^{\otimes n} \otimes I_m) |0\rangle_{n+m}$ being the amplitude of $|0\rangle_{n+m}$ at the end of the computation.

Restricted Weighted Sum. *For subsets $K \subseteq \{0, \ldots, N-1\}$ and $V \subseteq \{0, \ldots, M-1\}$, we can compute the restricted weighted sum,*

$$\sum_{\substack{k \in K \\ f(k) \in V}} w_k f(k),$$

by appropriate choices of f and h. In particular, value at risk (VaR) and comparators needed in option pricing can be computed using this pattern.

Note that Lebesgue integrals and partial moments of statistical distributions are also applications of the generalized inner product.

4 Quantum State Encoding of Common Distributions and Functions

Efficient quantum state preparation is critical for computing inner products. Specifically, encoding a normal distribution as a quantum state is computationally notoriously difficult [7]. The same is true for the preparation of quantum states with amplitudes or probabilities corresponding to the values of a linear function or higher order polynomial. For example, in [8] a value y of a polynomial is approximated by $y \approx \sin^2(y + \frac{\pi}{4}) - \frac{1}{2}$, which is equivalent to $y \approx \frac{1}{2}\sin(2y)$ for $y \approx 0$. Note that this is an approximation using probabilities. More details about this approximation are provided in the Appendix A.

In this section we introduce an exact encoding of the raised cosine distribution, in probabilities, (Section 4.1), which can approximate the normal distribution [9]. The implementation borrows an idea from Digital Signal Processing. We encode Fourier coefficients in the quantum state, and then apply the (inverse) quantum Fourier transform. We use this same idea to provide increasingly better approximations for the normal distribution for a given number of qubits using more Fourier coefficients in Section 4.2.

Additionally, in Section 4.3 we use a canonical encoding of a linear function using the approximation $y \approx \sin(y)$, for $y \approx 0$, in amplitudes instead of probabilities. We also provide a heuristic implementation for a high-precision approximation of identity as a canonical implementation of a linear function for three, four and five qubits (Section 4.4).

4.1 Exact Encoding of the Raised Cosine Distribution

In this section, we describe a quantum implementation of the raised cosine probability distribution. The probability density function for the raised cosine distribution is

$$\begin{aligned} p(x) &= \frac{1}{2\sigma}\left(1 + \cos\left(\frac{x-\mu}{\sigma}\pi\right)\right) \\ &= \frac{1}{\sigma}\cos^2\left(\frac{x-\mu}{2\sigma}\pi\right) \end{aligned} \tag{14}$$

for $\mu - \sigma \leq x \leq \mu + \sigma$.

Given a quantum system with n qubits, we start by encoding the state

$$\frac{1}{\sqrt{2}} |0\rangle_n - \frac{1}{\sqrt{2}} |2^{n-1}\rangle_n$$

then we apply the inverse quantum Fourier transform. The final circuit, denoted by $N_{1,n}$, is represented in Fig. 4. The resulting state is

$$\begin{aligned} |\nu_1\rangle_n &= N_{1,n}\,|0\rangle_n \\ &= \sqrt{\frac{2}{N}}\sum_{k=0}^{N-1}\sin(k\frac{\pi}{N})e^{i(\frac{\pi}{2}-k\frac{\pi}{N})}\,|k\rangle_n\,. \end{aligned} \tag{15}$$

The corresponding probability distribution is

$$\begin{aligned} p(k) &= \frac{2}{N}\cos^2\left(\left(k-\frac{N}{2}\right)\frac{\pi}{N}\right) \\ &= \frac{2}{N}\sin^2\left(k\frac{\pi}{N}\right) \end{aligned} \tag{16}$$

for $0 \leq k < N$. This matches the probability density function of the raised cosine distribution (Equation 14) when $\mu = \frac{N}{2}$ and $\sigma = \frac{N}{2}$.

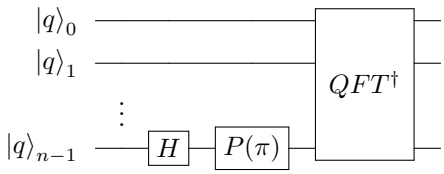

Figure 4: The quantum circuit that prepares the state with a raised cosine probablity distribution represented in Eq . 15, where n is a positive integer representing the number of qubits in the system, H is a single qubit Hadamard gate, P is a single qubit Phase gate.

4.2 Approximations of the Normal Distribution with Trigonometric Functions

The raised cosine distribution is an approximation for the normal distribution, as introduced by Raab and Green in [9].

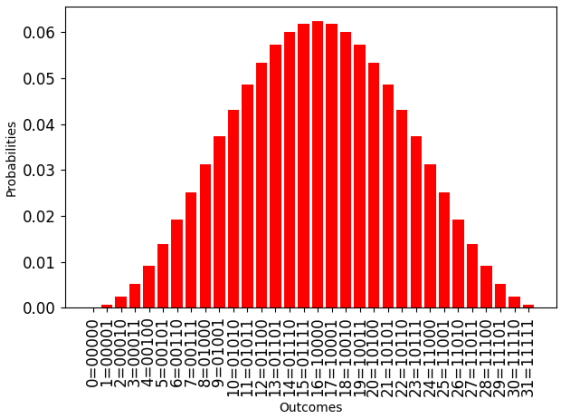

Figure 5: Outcome probability distribution after encoding the raised cosine distribution using a five-qubit quantum state.

We can use the equivalent of three Fourier coefficients instead of two and create the n-qubit quantum state:

$$\sqrt{\frac{2}{3}}\,|0\rangle_n - \frac{1}{\sqrt{6}}\,|2^{n-1}\rangle_n - \frac{1}{\sqrt{6}}\,|2^n-1\rangle_n \tag{17}$$

Then, we apply the inverse quantum Fourier transform. Let us denote the composite unitary operator by $N_{2,n}$:
The resulting state is

$$\begin{aligned} |\nu_2\rangle_n &= N_{2,n}\,|0\rangle_n \\ &= \sqrt{\frac{8}{3N}}\sum_{k=0}^{N-1}\sin^2\left(k\frac{\pi}{N}\right)|k\rangle_n\,. \end{aligned} \tag{18}$$

The corresponding probability distribution is

$$p(k) = \frac{8}{3N}\cos^4\left(\left(k-\frac{N}{2}\right)\frac{\pi}{N}\right) = \frac{8}{3N}\sin^4\left(k\frac{\pi}{N}\right) \tag{19}$$

for $0 \leq k < N$.
Note that:

$$\sum_{k=0}^{N-1}\sin^2\left(k\frac{\pi}{N}\right) = \frac{N}{2} \tag{20}$$

The resulting probability distribution is a better approximation of a normal distribution than the raised cosine probability distribution. Using more Fourier coefficients yields even better approximations.
We can use five Fourier coefficients to create the state

$$\begin{aligned} &\frac{6}{\sqrt{70}}\,|0\rangle_n + \frac{1}{\sqrt{70}}\,|2^{n-2}\rangle_n + \frac{1}{\sqrt{70}}\,|2^{n-1}-1\rangle_n \\ &\quad - \frac{4}{\sqrt{70}}\,|2^{n-1}\rangle_n - \frac{4}{\sqrt{70}}\,|2^n-1\rangle_n\,, \end{aligned} \tag{21}$$

followed by the inverse quantum Fourier transform. Let us denote the composite unitary operator by $N_{4,n}$:

$$\begin{aligned} |\nu_4\rangle_n &= N_{4,n}\,|0\rangle_n \\ &= \sqrt{\frac{128}{35N}}\sum_{k=0}^{N-1}\sin^4\left(k\frac{\pi}{N}\right)|k\rangle_n \end{aligned}$$

The corresponding probability distribution is

$$p(k) = \frac{128}{35N}\cos^8\left(\left(k-\frac{N}{2}\right)\frac{\pi}{N}\right) = \frac{128}{35N}\sin^8\left(k\frac{\pi}{N}\right) \tag{22}$$

for $0 \leq k < N$.

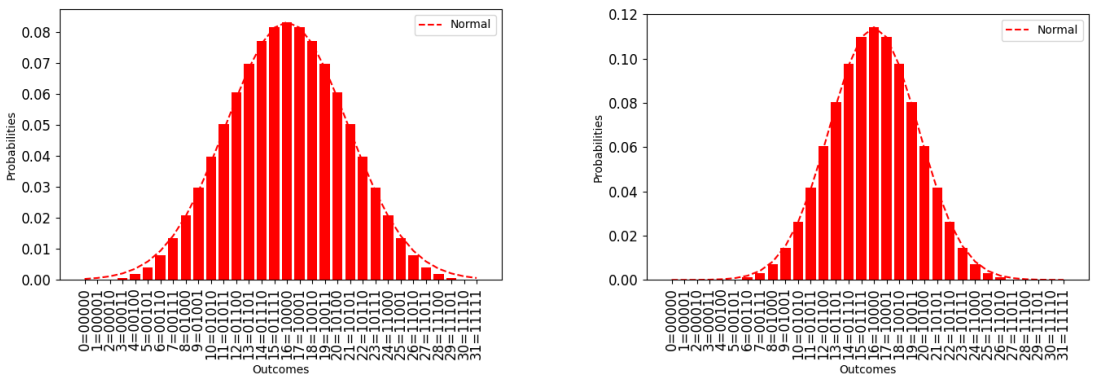

Figure 6: Left: The outcome probability distribution of the five-qubit state prepared using three Fourier coefficients (Eq. 17) resulting in the probability distribution in Eq . 19. The corresponding normal distribution is included for comparison. Right: The outcome probability distribution of the five-qubit state prepared using five Fourier coefficients (Eq . 21) resulting in the probability distribution in Eq. 22. The corresponding normal distribution is included for comparison.

4.3 Approximation of Linear Functions with Trigonometric Functions

Given a quantum system with n qubits and any $\theta \in \mathbb{R}$, let us denote by $T_{\theta,n}$ a unitary operator that creates the state

$$\begin{aligned} |\tau_\theta\rangle_{n+1} &= T_{\theta,n} |0\rangle_n \\ &= \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \sin(k\theta) |k\rangle_n |0\rangle_1 + \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \cos(k\theta) |k\rangle_n |1\rangle_1 \end{aligned} \tag{23}$$

For a small θ the amplitudes $\frac{1}{\sqrt{N}} \sin(k\theta)$ approximate the values of the linear function $k \mapsto \frac{\theta}{\sqrt{N}} k$ for $0 \leq k < N$.

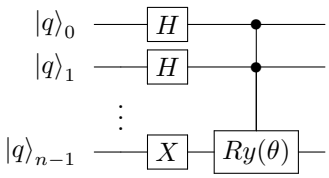

Figure 7: The quantum circuit that prepares the quantum state represented in Eq. 23.

4.4 Heuristic Implementation of Linear Functions

Given a number of qubits, non-canonical, heuristic search methods can be used to create arbitrarily close approximations of linear functions.

For an n qubit quantum system, with $N = 2^n$, let us denote by L_n a unitary operator that creates the state

$$\begin{aligned} |\lambda\rangle_n &= L_n |0\rangle_n \\ &= \sqrt{\frac{6}{(N-1)N(2N-1)}} \sum_{k=0}^{N-1} k |k\rangle_n . \end{aligned} \tag{24}$$

Visualizations of the quantum states $|\lambda\rangle_3$, $|\lambda\rangle_4$ and $|\lambda\rangle_5$ are in Appendix B.1.

5 Applications

In this section we are presenting applications of the Weighted Sum pattern 3.1 and Controlled Weighted Sum pattern 3.2 to expected value, value at risk, payoff, and value counting computations used in financial engineering and optimization.

Functions need to be discretized when encoded in quantum states. The Quantum Dictionary pattern allows to encode discrete functions represented as binary polynomials as described in Section 2.4. In [10] it is shown that the expected value of any linear function can be computed using a canonical linear function. Appendix A contains a short review of this approximation method.

We show that having an efficient implementation of a canonical linear function (e.g. the identity) allows the computation of the expected value of any discrete function. The implementation can be exact or approximate.

5.1 The Expected Value of Any Discrete Function

In this section we assume the availability of an operator B that prepares the state defined in Equation 24.

Given integers $n > 0$ and $m > 0$, with $N = 2^n$ and $M = 2^m$, and a function $f : \{0, \ldots, 2^n - 1\} \to \{0, \ldots, 2^m - 1\}$, assume we want to compute the weighted sum

$$\sum_{k=0}^{N-1} w_k f(k)$$

for weights $w_k = \sin^2(k\frac{\pi}{N})$ for $0 \leq k < N$.

Considering the representation of f as a binary polynomial $p : \{0,1\}^n \to \{0,1\}^m$, as described in Section 2.1, we need to compute the weighted sum

$$\sum_{k=0}^{N-1} w_k p(k).$$

As an example, with $n = 3$ and $m = 4$, we will show how to perform this computation for the binary polynomial

$$p(k_0, k_1, k_2) = 7 + 4k_1 - 5k_0k_1 - 2k_0k_2 \tag{25}$$

for $(k_0, k_1, k_2) \in \{0,1\}^n$, with $k = \sum_{j=0}^{n-1} k_j 2^j$ being the binary expansion of k as in Section 2.1.

Using the Controlled Weighted Sum pattern 3.2, where $A = N_{2,n}$ as described in Equation 18, and $B = L_m$ as described in Equation 24, $a = \sqrt{\frac{8}{3N}}$, and $b = \sqrt{\frac{6}{(M-1)M(2M-1)}}$ we obtain the result

$$\sum_{k=0}^{N-1} w_k f(k) = \sqrt{N}\sqrt{\frac{3N}{8}}\sqrt{\frac{(M-1)M(2M-1)}{6}} \cdot \langle 0| \left(H^{\otimes n} \otimes B^\dagger\right) F (A \otimes I_m) |0\rangle_{n+m} \tag{26}$$

Running the quantum computation in a simulator yields $\langle 0| \left(H^{\otimes n} \otimes B^\dagger\right) F (A \otimes I_m) |0\rangle_{n+m} = 0.17835$ (the amplitude of $|0\rangle_{n+m}$), and from Equation 26 we obtain $\sum_{k=0}^{N-1} w_k f(k) \approx 30.76777$. A direct classical calculation gives $\sum_{k=0}^{N-1} w_k f(k) \approx 30.76777$.

The results of performing this experiment on real quantum hardware can be found in Section 6.

5.2 Payoff Computation in Option Pricing

For simplicity we will use the same discrete function in Section 5.1 to represent a payoff function in an option pricing calculation, whose values are listed below.

k	$f(k)$
0	7
1	7
2	11
3	11
4	7
5	5
6	6
7	4

Assume that the strike price is $K = 7$, and the price distribution is represented by the values w_k. Then we are interested in computing

$$\sum_{\substack{0 \le k < N \\ f(k) \ge K}} w_k \left(f(k) - K\right)$$

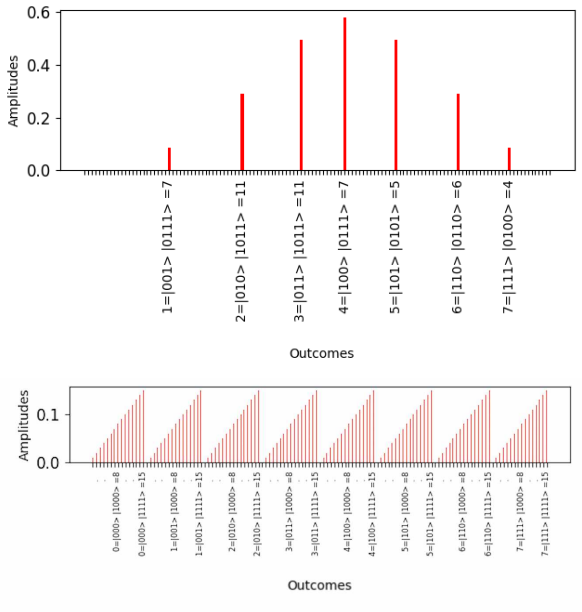

Figure 8: Top: Visualization of the amplitudes of a quantum system after applying the operator $A = N_{2,3}$, using 3 qubits for the key register and and 4 qubits for the value register. The key and value pairs on the x-axis show the result of encoding the binary polynomial in Eq. 25. The distribution of weights applied to the key register (as in Eq. 18) is an approximation of a normal distribution in the amplitudes. Bottom: Visualization of amplitudes of a quantum system, using 3 qubits for the key register and 4 qubits for the value register, after applying the operator $B = L_4$ and Hadamard gates on the key register. Encoding the linear function in the value register (as in Eq . 24) results in the identity function implemented in amplitudes, repeated for each key value.

We can do that by using the Controlled Weighted Sum pattern 3.2 with hashes

$$h_v = \begin{cases} v - K, & \text{if } v \ge K \\ 0, & \text{otherwise} \end{cases}$$

Alternatively, we can encode the function $f - K$ instead of f and choose hashes that are identity for non-negative inputs and 0 for negative ones. Note that the hash function acts as a value selector and replaces the comparator used in [10].

5.3 Value at Risk

Within the context and with the notations of the Weighted Sum pattern 3.1, for a given $0 \le l < N$, if $b = \frac{1}{\sqrt{l}}$ and

$$f(k) = \begin{cases} 1, & \text{if } k \le l \\ 0, & \text{otherwise} \end{cases}$$

then we can compute

$$\sum_{k=0}^{l} w_k.$$

This allows the computation of Value at Risk when the values w_k represent a price distribution by using a binary search over l as also described in [8].

Fig. 9 shows an example of a state that can be used to estimate the sum in Equation 5.3 for $N = 8$ and $l = 3$

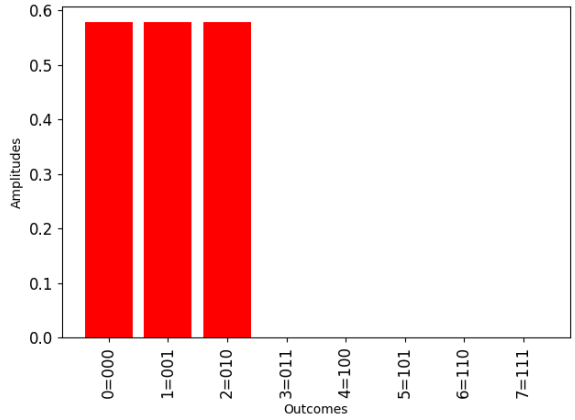

Figure 9: Visualization of the amplitudes of a three-qubit quantile quantum state, used in computing Value at Risk for $N = 8$ and $l = 3$.

Note that the Controlled Weighted Sum pattern 3.2 can also be used to estimate Value at Risk.

5.4 The Expected Value of a Non-Linear Function

Next, we consider the computation of the expected value of a non-linear function with a normally distributed argument.

Given an integer $n > 0$ and $N = 2^n$, assume we want to compute the weighted sum of a function f defined by $f(k) = r(x, y)$, for integers $0 \leq k < N$, where $k = 4x + y$, $0 \leq x, y < 3$:

$$\sum_{k=0}^{N-1} w_k f(k)$$

for weights $w_k = \sin^2(k\frac{\pi}{N})$ for $0 \leq k < N$, where r is the rational function defined by

$$r(x, y) = \frac{1}{7.856}\left(\frac{4.01 - x}{1 + x} + \frac{4.01 - 2y + x}{(1 + y)^2} - 0.344\right)$$

The rational function r is an example of a function that can be used in derivative pricing.

Using the Weighted Sum pattern 3.1, where $A = N_{2,4}$, as described in Equation 18, for $n = 4$ qubits, and B encodes normalized values of the function f, $a = \sqrt{\frac{8}{3N}}$ and $b = 1$, we obtain the result

$$\sum_{k=0}^{N-1} w_k f(k) = \sqrt{\frac{3N}{8}} \langle 0| B^\dagger A |0\rangle_n$$

The states prepared by operators A and B are shown in Fig. 10.

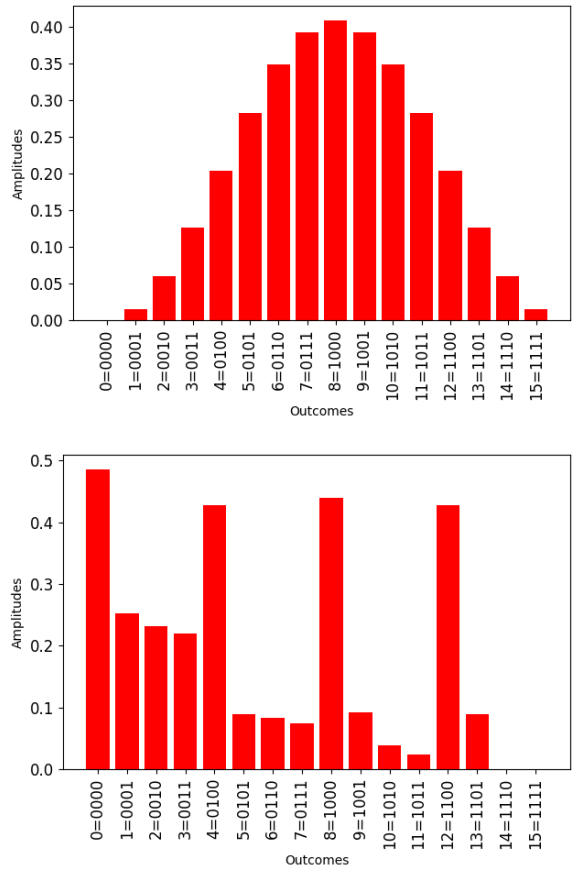

Figure 10: Top: Visualization of the amplitudes of a four-qubit quantum state prepared by the unitary operator A, as described in Eq. 18. Bottom: Visualization of the amplitudes of a four-qubit quantum state prepared by the unitary operator B, which encodes normalized values of the function f as described in Section 5.4.

Running the computation in a simulator yields $\langle 0| B^\dagger A |0\rangle_n \approx 0.55050$, leading to $\sum_{k=0}^{N-1} w_k f(k) \approx 1.34845$. A direct classical calculation gives $\sum_{k=0}^{N-1} w_k f(k) \approx 1.33431$.

5.5 The Expected Value of Linear Functions: Exact Version

5.5.1 Any Linear Function

Next, we consider the computation of the expected value of the identity function, with a normally distributed argument.

Given an integer $n > 0$ and $N = 2^n$, assume we want to compute the weighted sum of a function f defined by $f(k) = 1 + 2k$ for integers $0 \leq k < N$:

$$\sum_{k=0}^{N-1} w_k f(k)$$

for weights $w_k = \sin^2(k\frac{\pi}{N})$ for $0 \leq k < N$.

Using the Weighted Sum pattern 3.1, where $A = N_{2,3}$ as described in Equation 18, for $n = 3$ qubits,

$B = L_3$ as described in Equation 24, $a = \sqrt{\frac{8}{3N}}$ and $b = \frac{1}{\sqrt{\sum_{k=0}^{N-1}(1+2k)^2}}$, we obtain

$$\sum_{k=0}^{N-1} w_k k = \sqrt{\frac{3N}{8}} \sqrt{\sum_{k=0}^{N-1}(1+2k)^2} \langle 0| B^\dagger A |0\rangle_n .$$

The states prepared by operators A and B are shown in Fig. 11.

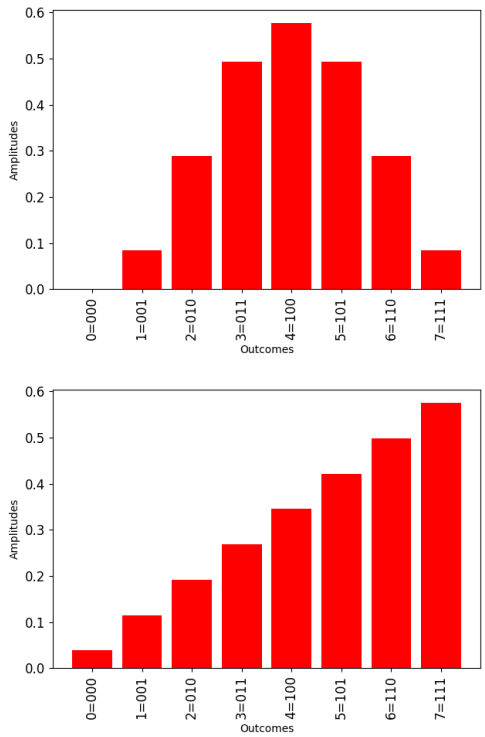

Figure 11: Top: Visualization of the amplitudes of a quantum state prepared by the unitary operator $A = N_{2,3}$, as described in Eq. 18, using three qubits. Bottom: Visualization of the amplitudes of a quantum state prepared by the unitary operator $B = L_3$, as described in Eq. 24, using three qubits.

As an example, for $n = 3$ and $N = 2^n$, running the computation in a simulator gives $\langle 0| B^\dagger A |0\rangle_n \approx 0.79705$, leading to $\sum_{k=0}^{N-1} w_k(1+2k) \approx 36.0$. A direct computation shows that $\sum_{k=0}^{N-1} w_k f(k) = 36.0$.

5.5.2 A Canonical Linear Function

Given an integer $n > 0$ and $N = 2^n$, assume we want to compute the weighted sum of the identity function f defined by $f(k) = k$ for integers $0 \le k < N$:

$$\sum_{k=0}^{N-1} w_k k$$

for weights $w_k = \sin^2(k\frac{\pi}{N})$ for $0 \le k < N$.

Using the Weighted Sum pattern 3.1, where $A = N_{2,3}$ as described in Equation 18, for $n = 3$ qubits, $B = L_3$ as described in Equation 24, $a = \sqrt{\frac{8}{3N}}$, and $b = \sqrt{\frac{6}{(N-1)N(2N-1)}}$, we obtain

$$\sum_{k=0}^{N-1} w_k k = \frac{N}{4}\sqrt{(N-1)(2N-1)} \langle 0| B^\dagger A |0\rangle_n$$

The states prepared by operators A and B are shown in Fig. 12.

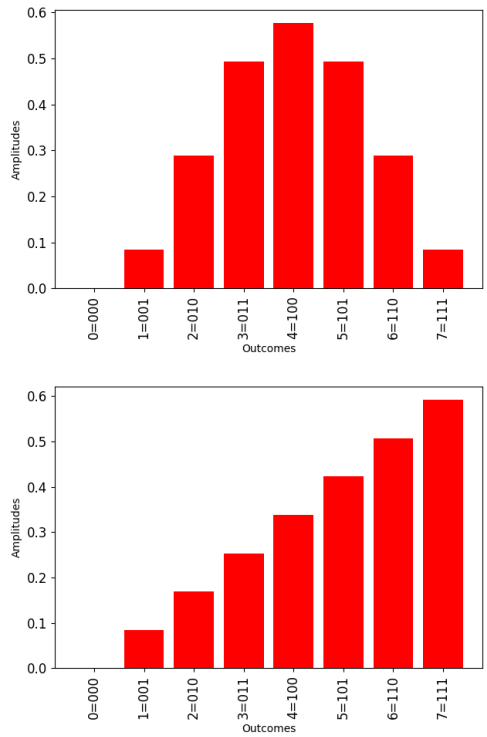

Figure 12: Top: Visualization of the amplitudes of a three-qubit quantum state prepared by the unitary operator A, as described in Eq. 18. Bottom: Visualization of the amplitudes of a three-qubit quantum state prepared by the unitary operator B, as described in Eq. 24.

Running the computation in a simulator gives $\langle 0| B^\dagger A |0\rangle_n \approx 0.77998$, leading to $\sum_{k=0}^{N-1} w_k k \approx 15.98493$.

Note that this canonical implementation of a linear functions allows for the computation of the expected value of any other linear function. For example, we can compute the weighted sum of the values of the function f defined by $f(k) = 1 + 2k$ for integers $0 \le k < N$:

$$\sum_{k=0}^{N-1} w_k f(k)$$

for weights $w_k = \sin^2(k\frac{\pi}{N})$ for $0 \le k < N$.

$$\sum_{k=0}^{N-1} w_k f(k) = \sum_{k=0}^{N-1} w_k(1+2k)$$
$$= \sum_{k=0}^{N-1} w_k + 2\sum_{k=0}^{N-1} w_k k = \frac{N}{2} + 2\sum_{k=0}^{N-1} w_k$$

For $n = 3$, and therefore $N = 8$, we obtain $\sum_{k=0}^{N-1} w_k(1+2k) \approx 4 + 2 \cdot 15.98493 = 35.96986$.

5.6 The Expected Value of Linear Functions: Approximate Version

This section is inspired by the method used in [10], except the approximation is done using amplitudes instead of probabilities.

As in Section 5.5, for a given an integer $n > 0$ and $N = 2^n$, assume we want to compute the weighted sum of the identity function f defined by $f(k) = k$ for integers $0 \le k < N$:

$$\sum_{k=0}^{N-1} w_k k$$

for weights $w_k = \sin^2(k\frac{\pi}{N})$ for $0 \le k < N$.

We will rely on the canonical approximation from Section 4.3.

Using the Weighted Sum pattern 3.1, where $A = N_{2,4}$, as described in Equation 18, and $B = T_{\frac{c}{2N},4}$, as described in Equation 23, for $n = 4$ and a value $c \approx 0$. Then $a = \sqrt{\frac{8}{3N}}$, $b = \sqrt{\frac{1}{N}}\frac{c}{2N}$, and we obtain

$$\sum_{k=0}^{N-1} w_k k = \frac{1}{\sqrt{\frac{8}{3N}}\sqrt{\frac{1}{N}}\frac{c}{2N}} \langle 0| B^\dagger A |0\rangle_{n+1}$$
$$= \sqrt{\frac{3}{2}}\frac{N^2}{c} \langle 0| B^\dagger A |0\rangle_{n+1}$$

The states prepared by operators A and B are shown in Fig. 14. The circuit will have n-qubits to encode the input $0 \le k < N$ and an ancillary qubit needed as target for the R_Y rotations in the operator B.

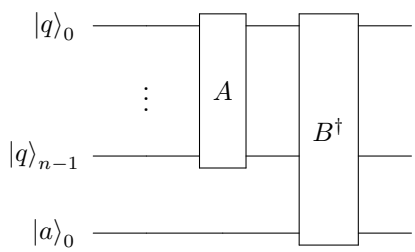

Figure 13: The quantum circuit that computes the inner product represented in Eq. 7 using unitary operators A and $B^\dagger$ (as defined in Eq. 8), and an ancillary qubit as the target for the R_y rotations in operator $B^\dagger$.

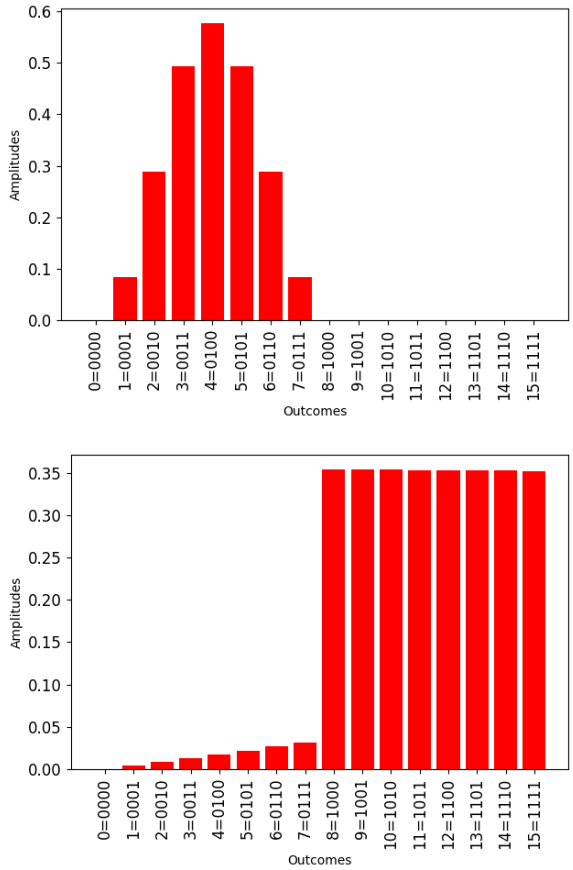

Figure 14: Top: Visualization of the amplitudes of a three-qubit quantum state prepared by the unitary operator A, as described in Eq. 18. Bottom: Visualization of the amplitudes of a three-qubit quantum state prepared by the unitary operator B, as described in Eq . 23, with a ancillary qubit added to the circuit to serve as the target for R_y rotations.

As an example, for $n = 3$, $N = 2^n$ and $c = 0.1$, when running the quantum circuit in a simulator we measure the amplitude of $|0\rangle_{n+1}$ as $\langle 0| B^\dagger A |0\rangle_{n+1} \approx 0.02041$ (it depends only on the distribution), leading to the approximation:

$$\sum_{k=0}^{N-1} w_k k = \sqrt{\frac{3}{2}}\frac{N^2}{c} 0.02041 = 15.99768.$$

Like in Section 5.5.2, we can use this canonical implementation to compute the expected value of any other linear function, e.g. f defined by $f(k) = 1 + 2k$ for integers $0 \le k < N$:

$$\sum_{k=0}^{N-1} w_k f(k)$$

A direct computation shows that $\sum_{k=0}^{N-1} w_k f(k) = 36.0$

Using the canonical computation above and the fact that $\sum_{k=0}^{N-1} w_k = \frac{N}{2}$ from Equation 20 we obtain $\sum_{k=0}^{N-1} w_k(1+2k) \approx 4 + 2 \cdot 15.99768 = 35.99536$.

5.7 Value Counting Using Generalized Inner Product

Given integers $n = 3$ and $m = 3$, $N = 2^n$ and $M = 2^m$, weights $w_k = 1$ for $0 \leq k < N$, hashes

$$h_v = \begin{cases} 1, & \text{if } v = v_0 \\ 0, & \text{otherwise} \end{cases}$$

for integers $0 \leq v < M$, and a function $f(k_0, k_1, k_2) = 2k_1 - k_0k_1 - 3k_0k_2$ for $(k_0, k_1, k_2) \in \{0,1\}^n$ and a value v_0, we want the number of inputs $k \in K$ such that $f(k) = v_0$.

k	$f(k)$
0	0
1	0
2	2
3	2
4	0
5	-3
6	1
7	-2

Therefore, we are interested in calculating the weighted sum of weighted/hashed function values:

$$\sum_{k=0}^{N-1} w_k h_{f(k)}$$

Note that we are using the binary representation of keys, i.e. $k = k_0 + 2k_1 + 4k_2$ for $0 \leq k < N$.

Let A be an operator that prepares the state $\sum_{k=0}^{N-1} a_k \left|k\right\rangle_n$, with $a = \frac{1}{\sqrt{N}}$ for $0 \leq k < N$, and let B be an operator that prepares the state $\left|v_0\right\rangle_m$, with $b = 1$ for $0 \leq v < M$, and let F be an operator that encodes the function f.

Using the Controlled Weighted Sum pattern 3.2, we can compute the weighted sum:

$$\begin{aligned} \sum_{k=0}^{N-1} w_k h_{f(k)} &= \left|f^{-1}(v_0)\}\right| \\ &= N \left\langle 0\right| (I_n \otimes B^\dagger) F (A \otimes I_m) \left|0\right\rangle_{n+m}. \end{aligned}$$

Running the computation in a simulator yields the value of $\left\langle 0\right| (I_n \otimes B^\dagger) F (A \otimes I_m) \left|0\right\rangle_{n+m}$ (i.e. the amplitude of $\left|0\right\rangle_{n+m}$) as 0.375, and therefore:

$$\left|f^{-1}(0)\}\right| = 8 \cdot 0.375 = 3$$

so the polynomial f has 3 zeros.

The quantum circuit that implements this computation as described above is shown in Fig. 15.

6 Experiments on Quantum Hardware

The experiments discussed in this section were run on IBM quantum devices powered by IBM Quantum Falcon Processors. For each experiment we include the best result of at least 10 runs.

As an experimental observation, we noticed a close correlation between the readout error on a quantum device and the error in inner product computations that does not generally apply to other computations. This may be an interesting area of research.

Expected value of discrete functions. In this experiment we performed the expected value computation discussed in Section 5.1 on the IBM ibmq_casablanca 7-qubit device with quantum volume 32. Each run of the experiment was performed with 8192 shots.

The amplitude of $\left|0\right\rangle$ in the best experiment result was ≈ 0.17747, compared to the simulated value ≈ 0.17835, as calculated in Section 5.1. The readout assignment error at the time of the experiment was 1.93%.

Expected value of linear functions for a trigonometric distribution. In this experiment we performed the expected value computation discussed in Section 5.5.2 on the IBM ibmq_jakarta 7-qubit device with quantum volume 16. Each run of the experiment was performed with 8192 shots.

The amplitude of $\left|0\right\rangle$ in the best experiment result was ≈ 0.72996, compared to the simulated value ≈ 0.77998, as calculated in Section 5.5.2. A comparison between the outcome probability distributions of the experiment and a simulation is shown in Fig. 16. The readout assignment error at the time of the experiment was 4.07%.

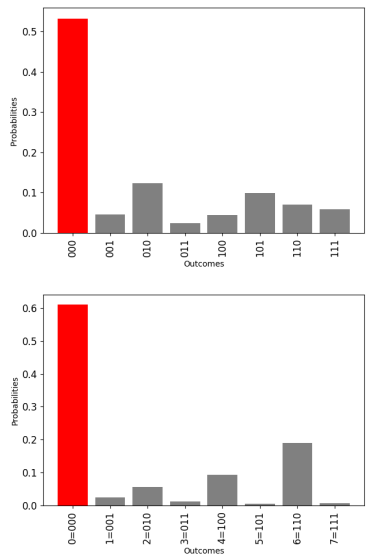

Figure 16: Visualization of the outcome probabilities of a three-qubit quantum state after performing the computation described in Section 5.5.2, running on ibmq_jakarta (top), and in a simulation environment (bottom). The outcomes different from $\left|0\right\rangle$ are grayed out because they do not contribute to the computation result.

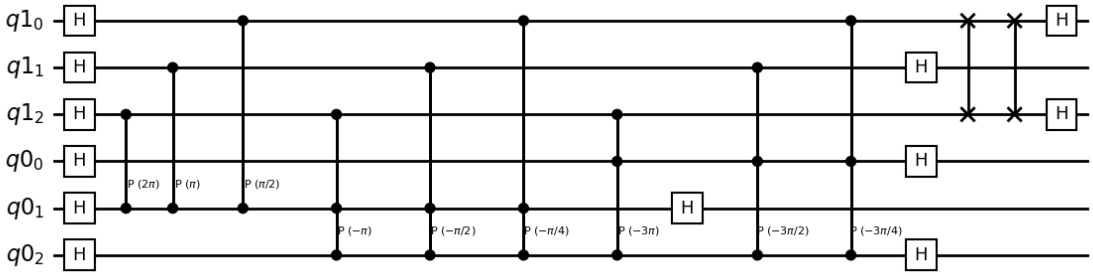

Figure 15: The quantum circuit visualizing the implementation of the example in Section 5.7 . The operators F and B both end with the inverse Fourier transform, which cancels out and therefore it can be omitted.

Expected value of linear functions for a normal distribution. In this experiment, we consider the computation of the expected value of the identity function with a normally distributed argument. Using the Weighted Sum pattern 3.1, where A is the unitary operator that encodes a normal distribution, as in Appendix B.2, and $B = L_n$ as described in Equation 24, for $n = 3$ qubits. The experiment was performed on the IBM ibmq_jakarta 7-qubit device with quantum volume 16. Each run of the experiment was performed with 8192 shots.

The amplitude of $|0\rangle$ in the best experiment result was ≈ 0.57378, compared to the simulated value ≈ 0.60131. The readout assignment error at the time of the experiment was 2.79%.

7 Related Work

Buhrman et al. [11] define a quantum inner product algorithm, the SWAP test, which efficiently solves the quantum states equality problem with high probability. This algorithm has been used as a building block in several clustering algorithms for supervised and unsupervised machine learning [12].

In the field of quantum deep learning, Tacchino et al. propose an inner product algorithm for implementing quantum neurons [13].

Woerner et al. propose using quantum algorithms for Monte Carlo simulations for financial risk management [8]. These algorithms effectively calculate the inner product of a quantum states representing the distribution of risk factors and asset values.

A widely used class of supervised learning algorithms is Support Vector Machines (SVMs) [14]. Their key computational component is a kernel function that calculates the inner product of feature vectors. This can be implemented by using a feature space implemented as a highly dimensional Hilbert space and a quantum inner product kernel function [15].

8 Concluding Remarks

Efficient methods for calculating expected values, variance, value at risk, etc. are very important in the financial industry.

Such methods rely on computing weighted sums. Finding quantum solutions in this space is an active area of research.

Most notably, the Woerner-Egger method addresses this family of computations by using Y-rotations to perform multiplication of amplitudes and measurement based addition for implementing weighted sums. In particular, the implementation of the expected value of a canonical linear function allows the calculation of the expected value of any linear function.

The Generalized Inner Product method introduced in this paper allows the calculation of the expected value of any discrete function, not necessarily linear, based on the encoding of a canonical linear function.

It also provides a canonical way to implement input function selection, and thus calculating value at risk (VaR), and a canonical way to implement output value selection, which enables straightforward implementations of comparators needed in option pricing [8, 10], and in general concentration metrics like counting the number of negative values a function takes, or the number of zeros of a function.

The paper provides additional building blocks like exact and approximate encodings for common statistical distributions (e.g. the raised cosine and normal distributions) and functions (e.g. linear functions). Examples of using exact or approximate encoding, and simple or generalized inner product are also included in the paper.

Even stronger generalizations of quantum inner products can be obtained by replacing unitary operators F and $H^{\otimes n}$ in Equation 13 with more general operators. This is an interesting future research area.

Acknowledgements

The authors want to thank Vitaliy Dorum for helping with the development of this manuscript.

The views expressed in this article are those of the authors and do not represent the views of Wells Fargo. This article is for informational purposes only. Nothing contained in this article should be construed as investment advice. Wells Fargo makes no express or implied warranties and expressly disclaims all legal, tax, and accounting implications related to this article.

We acknowledge the use of IBM Quantum services for this work. The views expressed are those of the authors, and do not reflect the official policy or position of IBM or the IBM Quantum team.

References

[1] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 212–219, 1996.

[2] Don Coppersmith. An approximate fourier transform useful in quantum factoring. Technical Report IBM Research Report RC19642, IBM, 1994.

[3] Austin Gilliam, Charlene Venci, Sreraman Muralidharan, Vitaliy Dorum, Eric May, Rajesh Narasimhan, and Constantin Gonciulea. Foundational patterns for efficient quantum computing, 2021. URL https://arxiv.org/abs/1907.11513.

[4] Austin Gilliam, Stefan Woerner, and Constantin Gonciulea. Grover adaptive search for constrained polynomial binary optimization. *Quantum*, 5:428, Apr 2021. ISSN 2521-327X. DOI: 10.22331/q-2021-04-08-428. URL http://dx.doi.org/10.22331/q-2021-04-08-428.

[5] IBM Quantum, 2021. URL https://quantum-computing.ibm.com/.

[6] Ryan O'Donnell. Analysis of boolean functions. *CoRR*, abs/2105.10386, 2021. URL https://arxiv.org/abs/2105.10386.

[7] Steven Herbert. No quantum speedup with grover-rudolph state preparation for quantum monte carlo integration. *Physical Review E*, 103(6), Jun 2021. ISSN 2470-0053. DOI: 10.1103/physreve.103.063302. URL http://dx.doi.org/10.1103/PhysRevE.103.063302.

[8] Stefan Woerner and Daniel J. Egger. Quantum risk analysis. *npj Quantum Information*, 5 (1), Feb 2019. ISSN 2056-6387. DOI: 10.1038/s41534-019-0130-6. URL http://dx.doi.org/10.1038/s41534-019-0130-6.

[9] David H Raab and Edward H Green. A cosine approximation to the normal distribution. *Psychometrika*, 26(4):447–450, 1961. ISSN 0033-3123.

[10] Nikitas Stamatopoulos, Daniel J. Egger, Yue Sun, Christa Zoufal, Raban Iten, Ning Shen, and Stefan Woerner. Option pricing using quantum computers. *Quantum*, 4:291, Jul 2020. ISSN 2521-327X. DOI: 10.22331/q-2020-07-06-291. URL http://dx.doi.org/10.22331/q-2020-07-06-291.

[11] Harry Buhrman, Richard Cleve, John Watrous, and Ronald de Wolf. Quantum fingerprinting. *Physical Review Letters*, 87 (16), Sep 2001. ISSN 1079-7114. DOI: 10.1103/physrevlett.87.167902. URL http://dx.doi.org/10.1103/PhysRevLett.87.167902.

[12] Seth Lloyd, Masoud Mohseni, and Patrick Rebentrost. Quantum algorithms for supervised and unsupervised machine learning, 2013.

[13] Francesco Tacchino, Chiara Macchiavello, Dario Gerace, and Daniele Bajoni. An artificial neuron implemented on an actual quantum processor. *npj Quantum Information*, 5(1), Mar 2019. ISSN 2056-6387. DOI: 10.1038/s41534-019-0140-4. URL http://dx.doi.org/10.1038/s41534-019-0140-4.

[14] Yunchao Liu, Srinivasan Arunachalam, and Kristan Temme. A rigorous and robust quantum speed-up in supervised machine learning. *CoRR*, abs/2010.02174, 2020. URL http://dblp.uni-trier.de/db/journals/corr/corr2010.html#abs-2010-02174.

[15] Maria Schuld and Nathan Killoran. Quantum machine learning in feature hilbert spaces. *Physical Review Letters*, 122(4), Feb 2019. ISSN 1079-7114. DOI: 10.1103/physrevlett.122.040504. URL http://dx.doi.org/10.1103/PhysRevLett.122.040504.

A The Woerner-Egger Approximation Method

Sum of Products. *Given an integer $n > 0$ and $N = 2^n$, probabilities $p_k \in [0,1]$ defined for integers $0 \leq k < N$, with $\sum_{k=0}^{N-1} p_k = 1$, and a function $f : \{0, \ldots, N-1\} \to [-1, 1]$ we are interested in calculating the sum of products*

$$\sum_{k=0}^{N-1} p_k f(k).$$

Solution Sketch. We paraphrase [8] in this description. We can use the approximation

$$y \approx \frac{1}{2}\sin(2y) = \sin^2(y + \frac{\pi}{4}) - \frac{1}{2} \text{ for } y \approx 0.$$

Then for a constant $c \approx 0$ we have

$$\begin{aligned}\sum_{k=0}^{N-1} p_k f(k) &\approx \sum_{k=0}^{N-1} p_k \frac{1}{2c}\sin(2cf(k)) = \sum_{k=0}^{N-1} p_k \frac{1}{c}\left(\sin^2\left(cf(k) + \frac{\pi}{4}\right) - \frac{1}{2}\right) \\ &= \frac{1}{c}\sum_{k=0}^{N-1} p_k \left(\sin^2\left(cf(k) + \frac{\pi}{4}\right)\right) - \frac{1}{2c} = \frac{1}{c}\sum_{k=0}^{N-1}\left(\sqrt{p_k}\sin\left(cf(k) + \frac{\pi}{4}\right)\right)^2 - \frac{1}{2c}\end{aligned}$$

The products $\sqrt{p_k}\sin(\theta_k)$ for $\theta_k \in \mathbb{R}$ can be encoded as amplitudes of a quantum states using R_Y single-qubit quantum gates acting on an ancillary qubit as described in [8]. In essence, given a quantum state that encodes the probability distribution p

$$\sum_{k=0}^{N-1} \sqrt{p_k}\,|k\rangle_n$$

the following quantum state can be built by adding an ancillary qubit and applying (controlled) R_Y gates to it

$$\sum_{k=0}^{N-1} \sqrt{p_k}\cos(\theta_k)\,|k\rangle_n\,|0\rangle + \sum_{k=0}^{N-1} \sqrt{p_k}\sin(\theta_k)\,|k\rangle_n\,|1\rangle\,,$$

where $\theta_k = cf(k) + \frac{\pi}{4}$.

The probability of measuring $|1\rangle$ in the ancillary qubit is

$$P_1 = \sum_{k=0}^{N-1} p_k \sin^2(\theta_k) \approx \sin^2(\frac{\pi}{4})\sum_{k=0}^{N-1} p_k = \frac{1}{2},$$

and it can be approximated using amplitude estimation algorithms.Then

$$\sum_{k=0}^{N-1} p_k f(k) \approx \frac{1}{2c}(2P_1 - 1).$$

□

Note that the summation is done at the probability level, through measurement, not in amplitudes.

The particular case of linear functions is treated in [10] where it is also pointed out that considering only the canonical linear function $f(k) = -1 + \frac{2}{N-1}k$ is sufficient.

Note that:

$$-1 \leq -1 + k\frac{2}{N-1} \leq 1 \text{ and } \frac{1}{2} - c \leq c\left(-1 + k\frac{2}{N-1}\right) + \frac{1}{2} \leq \frac{1}{2} + c.$$

Adding up the probabilities for outcomes with the ancillary qubit measured as $|1\rangle$, we get:

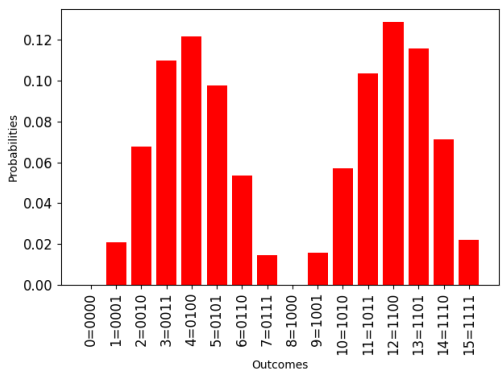

Figure 17: Outcome probability distribution of a quantum state that uses three main qubits and an ancillary qubit, prepared using the **Sum of Products** method.

$$
\begin{aligned}
P_1 &= \sum_{k=0}^{N-1} p_k \sin^2\left(c(-1+k\frac{2}{N-1})+\frac{\pi}{4}\right) \approx \sum_{k=0}^{N-1} p_k \left(c(-1+k\frac{2}{N-1})+\frac{1}{2}\right) \\
&= \left(\frac{1}{2}-c\right)\sum_{k=0}^{N-1} p_k + c\frac{2}{N-1}\sum_{k=0}^{N-1} p_k k = \frac{1}{2}-c+c\frac{2}{N-1}\sum_{k=0}^{N-1} p_k k
\end{aligned}
$$

Therefore

$$
\sum_{k=0}^{N-1} p_k k \approx \frac{N-1}{2c}\left(P_1-\frac{1}{2}+c\right).
$$

This can be used to calculate $\sum_{k=0}^{N-1} p_k f(k)$ for any linear function f by using its intercept and slope. Alternatively, as in [10], we can use the fact that a linear function is completely determined by its minimum and maximum, to obtain

$$
\sum_{k=0}^{N-1} p_k f(k) \approx m + \frac{M-m}{2c}\left(P_1-\frac{1}{2}+c\right).
$$

for a linear function $f:\{0,\dots,N-1\}\to[m,M]$.

Intuitively, around half (between $\frac{1}{2}-c$ and $\frac{1}{2}+c$) of each original amplitude is moved to the corresponding one that measures $|1\rangle$ in the ancilla.

B Heuristic Circuit Catalog

B.1 Encoding of the Identity Function

The following heuristic circuits provide implementations of a linear function, as discussed in Section 4.4.

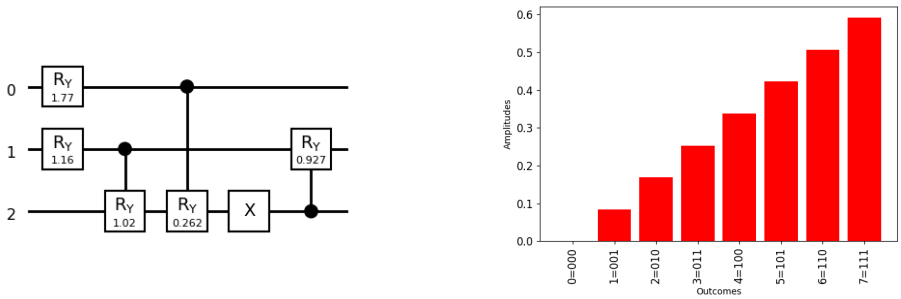

Figure 18: Left: The quantum circuit that encodes the quantum state $|\lambda\rangle_3$ using $n=3$ qubits, as defined in Eq . 24. Right: Visualization of the amplitudes of the quantum state with $n=3$ qubits, prepared using the circuit in the figure.

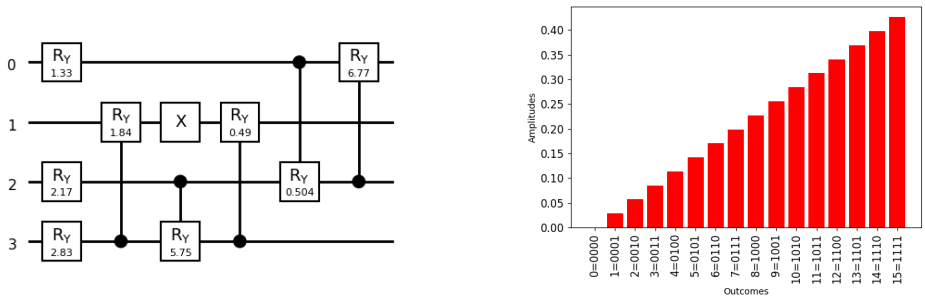

Figure 19: Left: The quantum circuit that encodes the quantum state $|\lambda\rangle_4$ using $n = 4$ qubits, as defined in Eq . 24. Right: Visualization of the amplitudes of a quantum state with $n = 4$ qubits, prepared using the circuit in the figure.

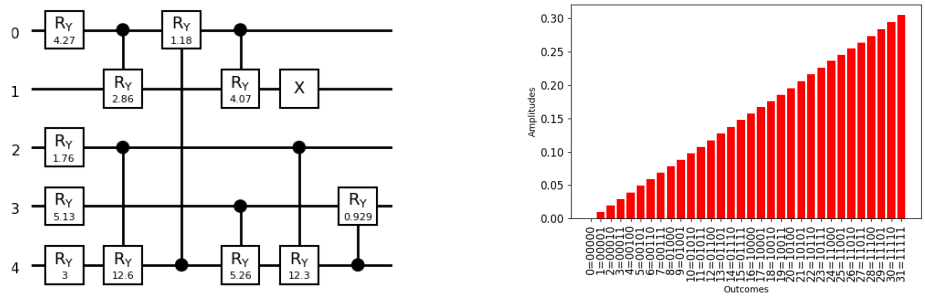

Figure 20: Left: The quantum circuit that encodes the quantum state $|\lambda\rangle_5$ using $n = 5$ qubits, as defined in Eq . 24. Right: Visualization of the amplitudes of a quantum state with $n = 5$ qubits, prepared using the circuit in the figure.

B.2 Encoding a Normal Distribution

The following heuristic circuits encode a normal distribution in a quantum state with three or four qubits, respectively.

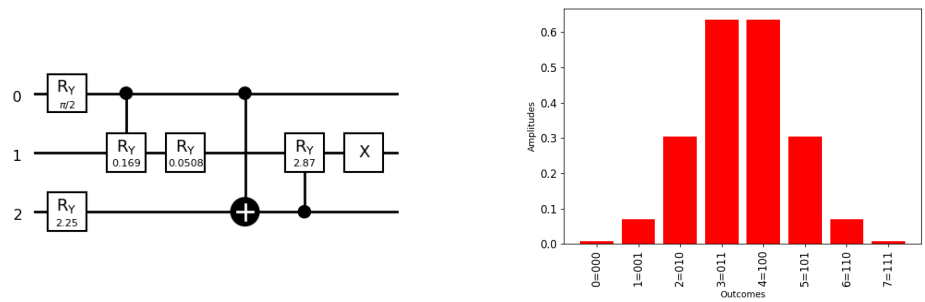

Figure 21: Left: The quantum circuit that encodes a normal distribution in a quantum state with three qubits. Right: Visualization of the amplitudes of a three-quantum state prepared using the circuit in the figure.

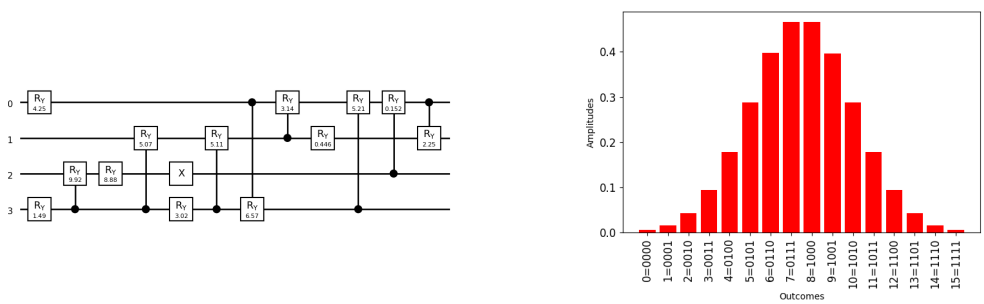

Figure 22: Left: The quantum circuit that encodes a normal distribution in a quantum state with four qubits. Right: Visualization of the amplitudes of a four-quantum state prepared using the circuit in the figure.